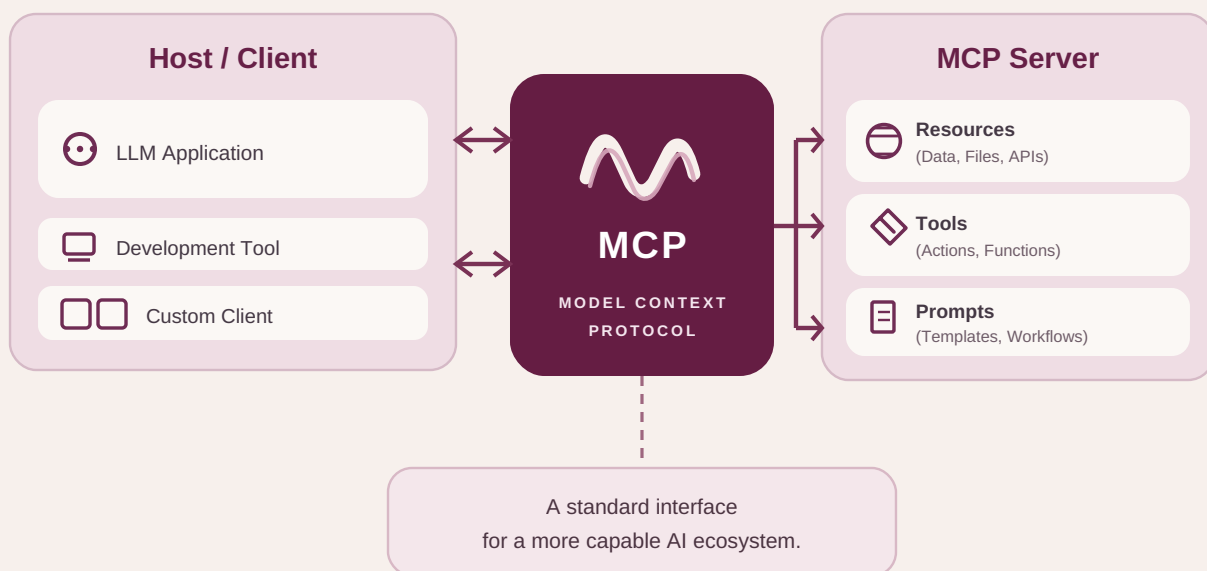


CHAPTER 02

The MCP Architecture

Hosts, clients, servers, primitives, and the JSON-RPC lifecycle that powers every MCP connection.



Premium Revision Notes

CONCEPTS

Clear Explanations

IMPLEMENTATION

Practical Code

DIAGRAMS

Visual Learning

INTERVIEW PREP

Key Questions

Chapter 02 — The MCP Architecture

In Chapter 1 we established *why* MCP exists: as AI tools multiplied, the context a model needed lived scattered across Jira, GitHub, MySQL, Drive, and Slack, and function calling forced every AI application to hand-write its own integration for every service — an $N \times M$ development nightmare. MCP's answer was to make the **service** responsible for its own integration once, so that any MCP-compatible AI application could use it without custom code.

This chapter is the *what* of MCP — the architecture, vocabulary, and message protocol that make that promise possible. We will build up the picture piece by piece: who talks to whom, what a server can offer, and the exact JSON messages that pass between them during a connection's lifetime.

1. Host, Client, and Server

MCP defines communication between exactly two kinds of entities: a **client** and a **server**. The client is simply your AI application — ChatGPT, Cursor, Claude Desktop, or a custom chatbot you build yourself. The server is the service you want that AI application to connect to — GitHub, Google Drive, Slack, or an internal company tool. The entire back-and-forth between these two speaks a single shared language: the Model Context Protocol.

IMPORTANT

"Client" and "server" here do not mean a browser and a backend in the web-development sense. The MCP *client* lives inside the AI application itself, and the MCP *server* is a small program that exposes one specific service to that application.

Zooming out one level, the AI application that houses one or more clients is called the **Host**. The Host is the actual program the user interacts with — Claude Desktop, an IDE, or a custom agent. Inside the Host, a separate MCP Client is created for every MCP Server the Host wants to talk to, and each client maintains a dedicated one-to-one connection with its server.

Why one client per server?

Each client-server pair maintains its own stateful session — its own capability negotiation, its own message IDs, and its own lifecycle. Isolating each connection behind a dedicated client means a failure or slowdown in one server (say, Slack) cannot corrupt the session state of another (say, GitHub). The Host is simply the coordinator that owns all of these client instances and routes the model's requests to the right one.

In practice, "Host" almost always refers to one of a small number of familiar programs:

Claude Desktop

Cursor

Windsurf

Custom agent / chatbot

IDE plugin

Whichever Host you use, adding a new server is a configuration change, not a coding task — you tell the Host how to launch or reach the server, and the Host creates and owns the corresponding client automatically.

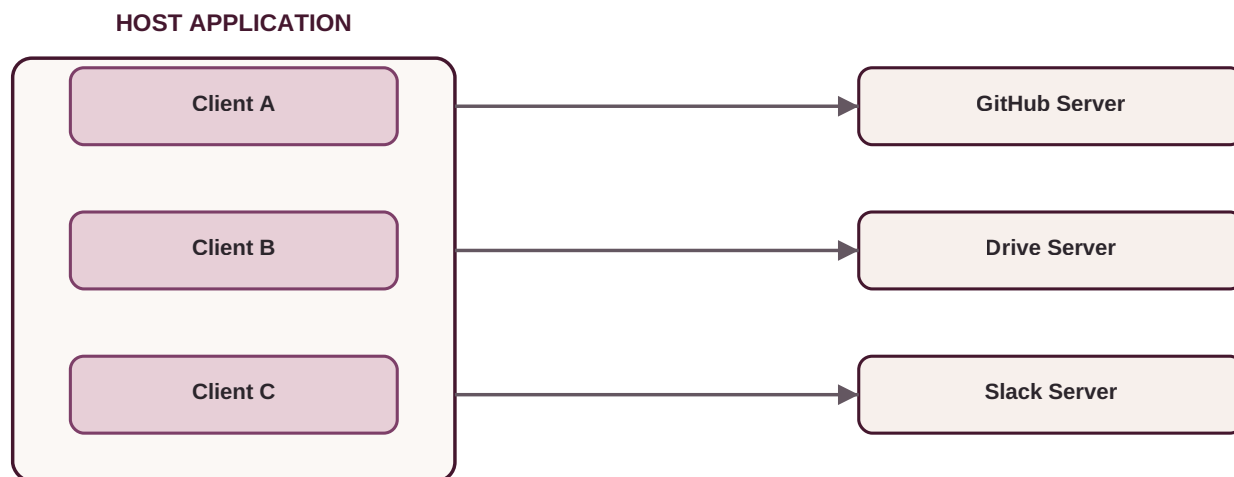


Figure 2.1 — A single Host runs one dedicated MCP Client per connected Server; each client-server pair speaks independently.

🔄 Quick Revision

- **Client** = the AI application's side of a single connection (built with the MCP Client SDK).
- **Server** = the service being connected to (built with the MCP Server SDK).
- **Host** = the overall AI application (e.g. Claude Desktop) that owns and coordinates multiple clients.
- Each Host-to-Server connection gets its own dedicated Client instance — never shared.

2. The Three Primitives

Every MCP Server can expose its capabilities to a client through exactly three standardized primitives. These are the vocabulary of MCP — any server, regardless of what service it wraps, describes what it offers using only these three categories.

Primitive	What it represents	Analogy
Tools	Actions the model can invoke, with side effects (e.g. create an issue, send a message)	A function call
Resources	Contextual data the model can read (e.g. a file, a database row, a document)	A GET endpoint
Prompts	Reusable, parameterized templates for common workflows	A saved query / macro

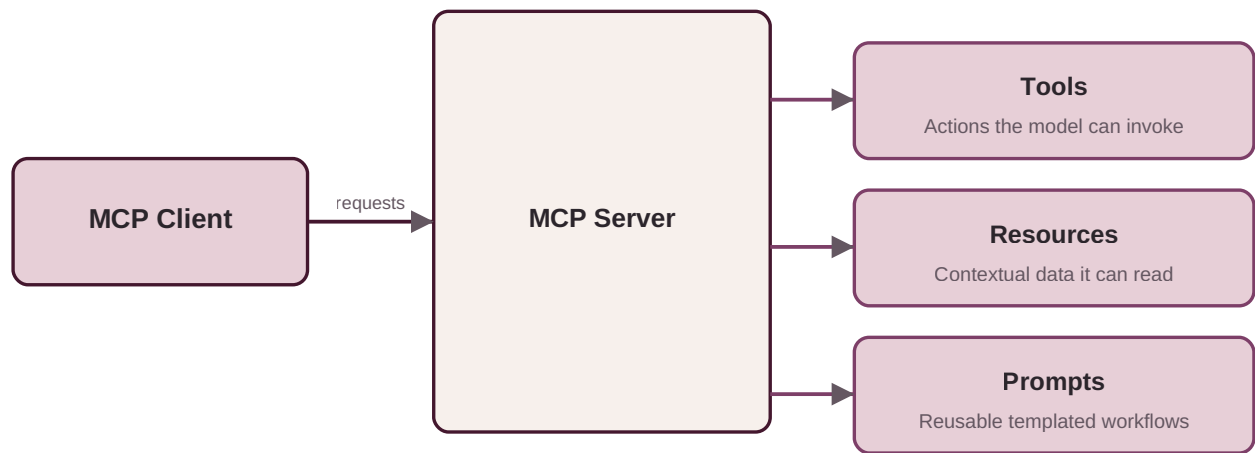


Figure 2.2 — A server advertises its *Tools*, *Resources*, and *Prompts*; the client requests whichever primitive the task needs.

Prompts in practice

Prompts are the least obvious of the three. A prompt is a named, structured template that produces a well-formed instruction from a small amount of input — the difference between a vague one-line ask and a fully specified request. Consider a GitHub server exposing an `issue_report_prompt`:

WITHOUT A PROMPT TEMPLATE

```
Title: Login bug
Body: The login button doesn't work.
```

ISSUE_REPORT_PROMPT OUTPUT

```
Title: Bug - Login Button Not Working

**Steps to Reproduce:**
1. Open the login page
2. Enter valid credentials
3. Click the Login button

**Expected Behavior:**
User should be logged in and redirected.

**Actual Behavior:**
Nothing happens when clicking Login.

**Environment:**
Chrome 121, macOS 14.2
```

COMMON MISTAKE

Treating prompts as optional decoration. A well-designed prompt primitive is what turns a vague user request into a structured, high-quality output — it standardizes how a task is done, not just what data is fetched.

Standard operations per primitive

Each primitive exposes a small, consistent set of JSON-RPC methods that the client uses to discover and use it. The pattern is always the same shape: a `list`-style method that returns what is available, followed by a `call`, `read`, or `get`-style method that actually uses one specific item from that list. A client never needs to guess a tool's name or a resource's shape in advance — it always discovers first.

Primitive	Methods	Purpose
Tools	tools/list	"What tools do you provide?"
	tools/call	"Run this tool with these arguments."
Resources	resources/list	"What resources are available?"
	resources/read	"Give me the content of this resource."
	resources/subscribe	"Notify me when this resource changes."
Prompts	prompts/list	"What prompt templates do you provide?"
	prompts/get	"Fetch a specific prompt template."

🔄 Quick Revision

- Tools = actions with side effects; Resources = readable context; Prompts = reusable templates.
- Every primitive follows a *list* then *use* pattern — discover what's available, then invoke it.
- Resources additionally support `subscribe` / `unsubscribe` for change notifications.

3. The Transport: JSON-RPC 2.0

All of this back-and-forth — listing tools, calling them, fetching resources — happens using a single, fixed message format: **JSON-RPC 2.0**. JSON-RPC combines the simplicity of JSON with the concept of a Remote Procedure Call, giving every request and response a standardized, predictable shape.

REQUEST

```
{
  "jsonrpc": "2.0",
  "method": "add",
  "params": [2, 3],
  "id": 1
}
```

RESPONSE

```
{
  "jsonrpc": "2.0",
  "result": 5,
  "id": 1
}
```

Figure 2.3 — A JSON-RPC request (left) and its matching response (right), linked by the shared `id` field.

Request and response fields

Field	Present in	Meaning
<code>jsonrpc</code>	Both	Always the literal string <code>"2.0"</code>
<code>method</code>	Request	The RPC method being called, e.g. <code>tools/call</code>
<code>params</code>	Request	Arguments for the method (object or array)
<code>id</code>	Both	Correlates a response to its originating request
<code>result</code>	Response (success)	The method's return value
<code>error</code>	Response (failure)	A structured error object

BEST PRACTICE

Because `id` correlates requests to responses, a client can have multiple in-flight requests to the same server and still match each reply correctly — treat the `id` as a required correlation key, not decoration.

The error object

When a request fails, the response replaces `result` with a structured `error` object:

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "error": {
    "code": -32601,
    "message": "Method not found",
    "data": { "extra": "optional info" }
  }
}
```

`code` is a machine-readable integer, `message` is a short human-readable explanation, and `data` is an optional field for extra debugging context. MCP reuses a set of standard JSON-RPC error codes:

Code	Name	Meaning
-32601	Method not found	Called a method the server never advertised
-32602	Invalid params	Request sent with wrong or missing parameters
-32600	Invalid Request	Missing required fields like <code>jsonrpc</code> or <code>method</code>
-32700	Parse error	Request body is not valid JSON
-32000+	Server-defined	Custom errors defined by the implementation (auth failure, rate limit, etc.)

🔄 Quick Revision

- JSON-RPC 2.0 = JSON + Remote Procedure Calls, in a fixed request/response shape.
- `id` links every response back to the request that triggered it.
- Errors always use the `{code, message, data}` shape instead of `result`.
- Codes below -32000 are reserved by the JSON-RPC spec; server-specific errors use -32000 and below custom ranges.

4. Local vs Remote Servers

An MCP server can run in one of two places, and the transport mechanism changes accordingly. A **local server** runs as a subprocess on the same machine as the Host and is reached over standard input/output (STDIO) — no network stack involved. A **remote server** runs elsewhere on the network or the internet and is reached over HTTP, with Server-Sent Events (SSE) used to stream messages back to the client.

	Local Server	Remote Server
Runs on	Your own machine	Another machine, over the network
Transport	STDIO (standard input/output)	HTTP + Server-Sent Events (SSE)
Typical use	Filesystem access, local dev tools	GitHub, Slack, Google Drive

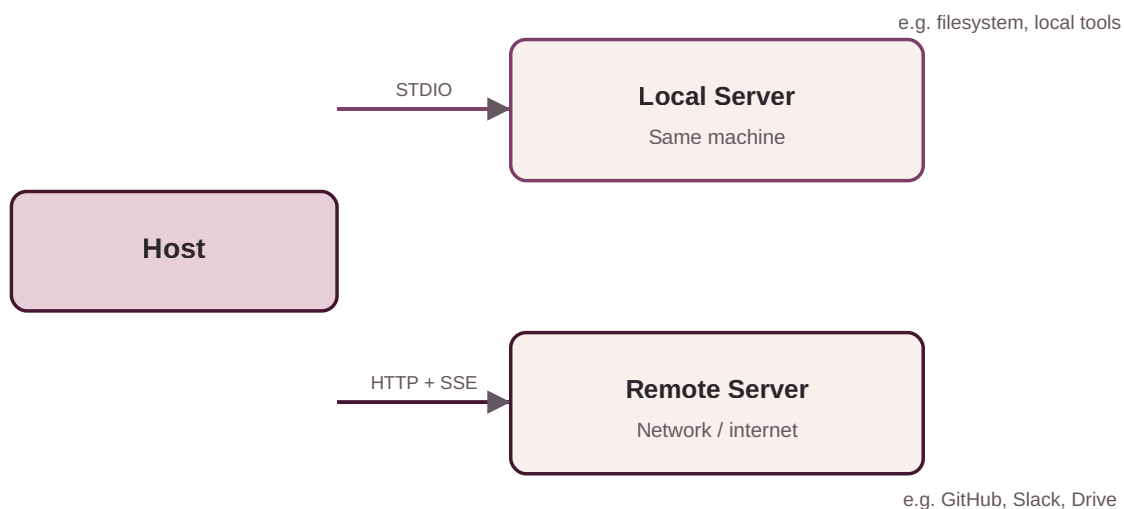


Figure 2.4 — Local servers communicate over STDIO on the same machine; remote servers communicate over HTTP with Server-Sent Events.

REAL WORLD INSIGHT

Claude Desktop's MCP configuration file is exactly where this choice becomes concrete: a local filesystem server is launched as a subprocess and spoken to over STDIO, while a remote GitHub server is reached over HTTP using a URL and an access token — both wrapped in the identical JSON-RPC message format underneath.

5. The MCP Lifecycle

Every MCP connection — regardless of transport — passes through three well-defined stages.

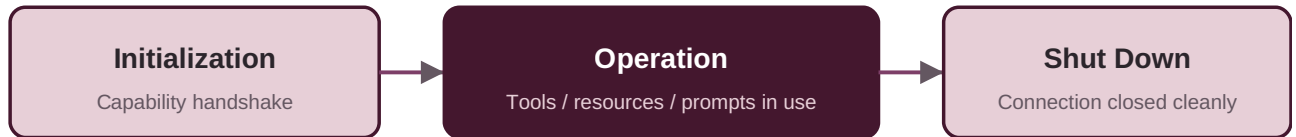


Figure 2.5 — The three stages of an MCP connection: a one-time handshake, the working phase, and a clean shutdown.

Stage 1 — Initialization

Before any tool can be called or resource read, the client and server perform a capability handshake in three steps.

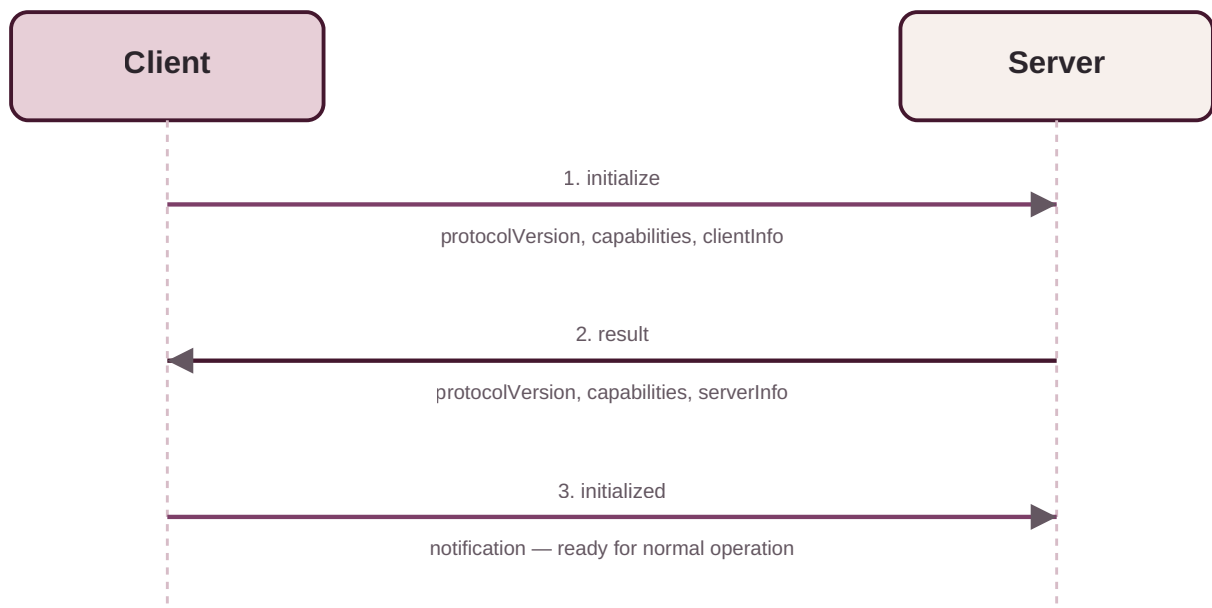


Figure 2.6 — The initialization handshake: the client proposes a protocol version and capabilities, the server responds with its own, and the client confirms readiness.

Step 1 of the handshake, spelled out as an actual JSON-RPC message, looks like this:

```
// Step 1 – Client sends initialize
{
  "jsonrpc": "2.0", "id": 1, "method": "initialize",
  "params": {
    "protocolVersion": "2025-03-26",
    "capabilities": { "roots": { "listChanged": true }, "sampling": {} },
    "clientInfo": { "name": "IDEPlugin", "version": "1.0.0" }
  }
}
```

IMPORTANT

Step 1 also performs **version negotiation**: the client proposes a protocol version (e.g. 2025-03-26). If the server does not support it, the server responds with the latest version it does support from its own supported list, and the client must decide whether to proceed or disconnect.

Stage 2 — Operation

Once `notifications/initialized` is sent, the connection enters normal operation. This is where the standard operations from Section 2 happen: `tools/list`, `tools/call`, `resources/read`, and so on, exchanged for as long as the session is active.

Stage 3 — Shut Down

The connection is closed cleanly by whichever side initiates termination — typically the client, when the Host application closes or the user disconnects the server.

🔄 Quick Revision

- Lifecycle stages: **Initialization** → **Operation** → **Shut Down**.
- Initialization is a three-step handshake: `initialize` request → `result` response → `notifications/initialized`.
- Version negotiation happens during initialization — the server may downgrade to a version it supports.
- No tool/resource/prompt operations are valid before initialization completes.

6. Capability Negotiation

The `capabilities` object exchanged during initialization is not just a formality — it establishes exactly which optional protocol features will be available for the rest of the session. Each side only offers what it actually supports.

Side	Capability	Sub-capabilities
Client	roots	listChanged
Client	sampling	—
Client	elicitation	—
Server	prompts	listChanged
Server	resources	listChanged, subscribe
Server	tools	listChanged
Server	logging	—

A `listChanged` sub-capability means that side promises to send a notification if its list of tools, resources, or prompts changes at runtime, so the other side doesn't have to keep re-polling.

Discovering and calling tools

Once initialized, the client typically calls `tools/list` to learn what the server offers, then `tools/call` with a specific tool name and arguments:

TOOLS/LIST RESPONSE

```
{
  "result": { "tools": [
    { "name": "listRepos", ... },
    { "name": "getFile", ... },
    { "name": "searchCode", ... },
    { "name": "createIssue", ... }
  ] }
}
```

TOOLS/CALL REQUEST

```
{
  "method": "tools/call",
  "params": {
    "name": "getFile",
    "arguments": {
      "owner": "org", "repo": "mcp-examples",
      "path": "README.md"
    }
  }
}
```

🔄 Quick Revision

- Capabilities are negotiated once, during initialization, and fixed for the session.
- `listChanged` lets a side push updates instead of requiring polling.
- Discovery (`*/list`) always precedes use (`*/call`, `*/read`, `*/get`).

7. Utility Messages: Pings, Progress, and Cancellation

Beyond the core primitives, MCP defines a small set of utility messages that keep long-running or unreliable connections healthy.

PING

A lightweight request/response used to check whether the other side is still alive and responsive:

```
{ "jsonrpc": "2.0", "id": 42, "method": "ping" }
{ "jsonrpc": "2.0", "id": 42, "result": {} }
```

PROGRESS NOTIFICATIONS

For a long-running tool call, the server can stream progress updates tied to a `progressToken` supplied by the client in the original request's `_meta` field:

```
{ "jsonrpc": "2.0", "method": "notifications/progress",
  "params": { "progressToken": "tok-7", "progress": 60, "total": 100,
             "message": "Searching 600 of 1000 files" } }
```

CANCELLATION

Either side can abandon an in-flight request by sending a cancellation notification referencing the original request's `id`:

```
{ "jsonrpc": "2.0", "method": "notifications/cancelled",
  "params": { "requestId": "7", "reason": "Timeout exceeded (30s)" } }
```

BEST PRACTICE

Always attach a `progressToken` when calling a tool you expect to run long (large searches, file indexing) — without it, the server has no channel to report progress, and the client is left blind until the final response arrives.

8. Choosing a Client and Server Strategy

When building an MCP-based system, there are two independent decisions: which client to use, and which servers to connect it to.

Client choice	Server choice
Ready-made client (Claude Desktop, Cursor)	Already-existing servers (GitHub, Drive, Slack)
Ready-made client, extended with config	Your own custom MCP server
A fully custom MCP client you build	Your own custom MCP server

Most teams start with a ready-made client such as Claude Desktop and connect it to a mix of existing official servers and one or two custom servers for internal tools — this needs zero client-side code, only a configuration file listing which servers to launch and how to reach them.

🔄 Quick Revision

- Client and server choice are independent — mix ready-made and custom as needed.
- Ready-made clients require no code, only a JSON configuration entry per server.
- Custom servers are only necessary for internal or proprietary tools without an official MCP server yet.

Key Takeaways

- MCP defines two roles — **Client** and **Server** — coordinated inside a **Host** application, with one dedicated client per connected server.
- Servers expose capabilities through exactly three primitives: **Tools** (actions), **Resources** (readable data), and **Prompts** (reusable templates).
- Every message on the wire is **JSON-RPC 2.0** — a fixed request/response shape correlated by `id`, with a standardized error object for failures.
- Servers are either **local** (STDIO, same machine) or **remote** (HTTP + SSE, network).
- Every connection passes through **Initialization** → **Operation** → **Shut Down**, with a three-step handshake (`initialize` → `result` → `initialized`) establishing protocol version and capabilities up front.
- Utility messages — **ping**, **progress**, and **cancellation** — keep long-running sessions observable and controllable.
- Capability negotiation during initialization means both sides only ever see features the other side actually supports — no client ever calls a method the server never advertised.
- This architecture is what makes Chapter 1's promise real: a service builds **one** MCP server, and every MCP-compatible Host — Claude Desktop, Cursor, or a custom agent — can use it without writing custom integration code.

Revision Sheet

Roles

Host owns Clients; each Client ↔ one Server; both speak MCP.

Primitives

Tools = actions · Resources = data · Prompts = templates.

Transport

Every message is JSON-RPC 2.0: method/params/id → result or error.

Server location

Local → STDIO. Remote → HTTP + SSE.

Lifecycle

initialize → result → initialized, then Operation, then Shut Down.

Utility messages

ping (liveness), progress (long tasks), cancellation (abandon a request).

Interview Questions

Q1. What is the difference between the Host, the Client, and the Server in MCP terminology?

Q2. Name the three MCP primitives and give a one-line description of each.

Q3. Why does MCP use JSON-RPC 2.0 rather than plain REST for its message format?

Q4. What are the three stages of the MCP connection lifecycle, and what happens in each?

Q5. What is the purpose of the `initialize` / `result` / `initialized` three-step handshake?

Q6. What is the difference between a local and a remote MCP server, transport-wise?

Q7. What does the `listChanged` sub-capability let a client or server avoid doing?

Q8. How does a client cancel a long-running tool call, and how does it track progress before that?

Further Reading

- Model Context Protocol specification — official protocol reference for lifecycle, primitives, and transports.
- JSON-RPC 2.0 specification — the exact request/response/error format MCP builds on.
- Anthropic's MCP SDK documentation — client and server SDKs referenced in Chapter 3's implementation walkthrough.